Practical 1

AIM:
Write a Python program to manage the borrowing records of books in a library and compute the average, highest/lowest borrowings, zero-borrow count, and the mode of the borrow counts.

PROBLEM STATEMENT:
To write a simple Python program that manages the borrowing records of books in a library and implements the following functionalities:
(i) compute the average number of books borrowed by all library members,
(ii) find the book with the highest and lowest number of borrowings,
(iii) count the number of members/books that have not been borrowed at all (borrow count of 0), and
(iv) display the most frequently borrowed count (the mode of the borrow counts).

OBJECTIVE:

1. To understand how Python lists are used to store simple real-world records.
2. To learn how to use loops to process the values stored in a list.
3. To compute simple statistics such as average, maximum, minimum and mode using basic Python code.
4.

OUTCOME:

1. Students will be able to store and traverse borrowing records using Python dictionaries.
2. Students will be able to compute simple statistical measures such as average and mode using basic loops.
3.

SOFTWARE / HARDWARE USED:

- Programming Language: Python 3.x
- IDE: VS Code / PyCharm / Google Colab / Jupyter Notebook
-

THEORY:

Dictionary: Dictionaries are used to store data values in key:value pairs.
A dictionary is a collection which is ordered*, changeable and do not allow duplicates.
Dictionaries are written with curly brackets, and have keys and values:
Create and print a dictionary:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(thisdict)
```

output:

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}
```

Accessing Items
You can access the items of a dictionary by referring to its key name, inside square brackets:

Example
Get the value of the "model" key:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
x = thisdict["model"]
```

output:

```
Mustang
```

Adding Items

Adding an item to the dictionary is done by using a new index key and assigning a value to it:
Example

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
thisdict["color"] = "red"
print(thisdict)
```

output

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964, 'color': 'red'}
```

Loop Through a Dictionary

You can loop through a dictionary by using a for loop.
When looping through a dictionary, the return value are the *keys* of the dictionary, but there are methods to return the *values* as well.
Example
Print all key names in the dictionary, one by one:

```
thisdict =        {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
for x in thisdict:
  print(x)
```

output

```
brand
model
year
```

Algorithm used:

STEP 1: Sample Data

Dictionary: member_name -> number of books borrowed
member_borrow_counts = {}

Dictionary: book_title -> number of times borrowed
book_borrow_counts = {}

STEP 2: Average number of books borrowed by members
def calculate_average_borrowed(members)

STEP 3: Find book with highest and lowest borrowings
def find_highest_and_lowest_book(books)

STEP 4: Count members who borrowed 0 books
def count_zero_borrowers(members)

STEP 5: Find most frequently borrowed book (mode)
def find_most_borrowed_book(books)

STEP 6: Run the program and display results

Input :

```
member_borrow_counts = {
   "Alice": 5,
   "Bob": 0,
   "Charlie": 3,
   "David": 0,
   "Eva": 8,
   "Frank": 3,
   "Grace": 1
}


# Dictionary: book_title -> number of times borrowed
book_borrow_counts = {
   "Python Basics": 12,
   "Data Structures 101": 7,
   "The Great Novel": 20,
   "History of Everything": 2,
   "Cooking for Beginners": 20
}
```

Output:

===== LIBRARY BORROWING REPORT =====

1. Average number of books borrowed per member: 2.86

2. Book with HIGHEST borrowings: The Great Novel (20 times)
Book with LOWEST borrowings: History of Everything (2 times)

3. Number of members who borrowed 0 books: 2

4. Most frequently borrowed book(s): ['The Great Novel', 'Cooking for Beginners'] with 20 borrowings

==========

Conclusion:
By this way, we can manage the borrowing records of books in a library

----------program with output printout ----------

```
"""
Library Borrowing Records Manager
A simple beginner-level program to analyze book borrowing data.
"""
# ------------------------------------------------
# STEP 1: Sample Data
# ------------------------------------------------

# Dictionary: member_name -> number of books borrowed
member_borrow_counts = {
    "Alice": 5,
    "Bob": 0,
    "Charlie": 3,
    "David": 0,
    "Eva": 8,
    "Frank": 3,
    "Grace": 1
}

# Dictionary: book_title -> number of times borrowed
book_borrow_counts = {
    "Python Basics": 12,
    "Data Structures 101": 7,
    "The Great Novel": 20,
    "History of Everything": 2,
    "Cooking for Beginners": 20
}
```

```
# ---------------------------------------------------
# STEP 2: Average number of books borrowed by members
# ---------------------------------------------------

def calculate_average_borrowed(members):
    total_borrowed = 0
    number_of_members = 0

    for member in members:
        total_borrowed = total_borrowed + members[member]
        number_of_members = number_of_members + 1

    average = total_borrowed / number_of_members
    return average


# ---------------------------------------------------
# STEP 3: Find book with highest and lowest borrowings
# ---------------------------------------------------

def find_highest_and_lowest_book(books):
    highest_book = None
    lowest_book = None
    highest_count = -1
    lowest_count = None

    for book in books:
        count = books[book]

        # Check for highest
        if count > highest_count:
            highest_count = count
            highest_book = book

        # Check for lowest
        if lowest_count is None or count < lowest_count:
            lowest_count = count
```

```
            lowest_book = book

    return highest_book, highest_count, lowest_book, lowest_count


# ------------------------------------------------
# STEP 4: Count members who borrowed 0 books
# ------------------------------------------------

def count_zero_borrowers(members):
    zero_count = 0

    for member in members:
        if members[member] == 0:
            zero_count = zero_count + 1

    return zero_count


# ------------------------------------------------
# STEP 5: Find most frequently borrowed book (mode)
# ------------------------------------------------

def find_most_borrowed_book(books):
    highest_count = -1
    most_borrowed_books = []

    # First, find the highest borrow count
    for book in books:
        if books[book] > highest_count:
            highest_count = books[book]

    # Then, collect all books that have that count
    # (in case there is a tie)
    for book in books:
        if books[book] == highest_count:
            most_borrowed_books.append(book)

    return most_borrowed_books, highest_count
```

```
# ------------------------------------------------
# STEP 6: Run the program and display results
# ------------------------------------------------

print("===== LIBRARY BORROWING REPORT =====\n")

# 1. Average books borrowed
average = calculate_average_borrowed(member_borrow_counts)
print("1. Average number of books borrowed per member:", round(average, 2))

# 2. Highest and lowest borrowed books
highest_book, highest_count, lowest_book, lowest_count =
find_highest_and_lowest_book(book_borrow_counts)
print("\n2. Book with HIGHEST borrowings:", highest_book, "(" + str(highest_count) + " times)")
print("   Book with LOWEST borrowings:", lowest_book, "(" + str(lowest_count) + " times)")

# 3. Members who borrowed 0 books
zero_borrowers = count_zero_borrowers(member_borrow_counts)
print("\n3. Number of members who borrowed 0 books:", zero_borrowers)

# 4. Most frequently borrowed book (mode)
most_borrowed_books, mode_count = find_most_borrowed_book(book_borrow_counts)
print("\n4. Most frequently borrowed book(s):", most_borrowed_books, "with", mode_count,
"borrowings")

print("\n====================================")
```

Practical 2

AIM:

Write a Python program to implement Linear Search and Binary Search to check whether a given customer account ID exists in a list, and compare the efficiency of the two techniques.

PROBLEM STATEMENT:

In an e-commerce system, customer account IDs is stored in a list. Write a Python program that implements

(i) Linear Search to check if a particular customer account ID exists in the list, and

(ii) (ii) Binary Search on the sorted list to check existence, thereby improving search efficiency over the basic linear search.

OBJECTIVE:

1. To understand the concept and need for searching techniques in data structures.
2. To implement Linear Search and Binary Search algorithms in Python.
3. To compare the time complexities of Linear Search and Binary Search.

OUTCOME:

1. Students will be able to implement Linear Search and Binary Search algorithms.
2. Students will be able to analyze and compare the efficiency of searching techniques for different data sizes.

SOFTWARE / HARDWARE USED:

- Programming Language: Python 3.x
- IDE: VS Code / PyCharm / Google Colab / Jupyter Notebook

THEORY:

1. Searching:

Searching is the process of finding whether a particular element (the key) is present in a given collection of data, and if present, locating its position.

2. Linear Search:

Linear (sequential) search examines every element of the list one by one, starting from the first, and compares it with the target value until a match is found or the list is exhausted. It works on both sorted and unsorted lists and has a worst-case time complexity of O(n).

3. Binary Search:

Binary search is a divide-and-conquer algorithm that operates only on a sorted list. It repeatedly compares the target value with the middle element of the current search range and eliminates half of the remaining elements at every step, giving it a time complexity of O(log n).

Binary Search

"Binary search is an searching algorithm which is used to find element from the sorted list"

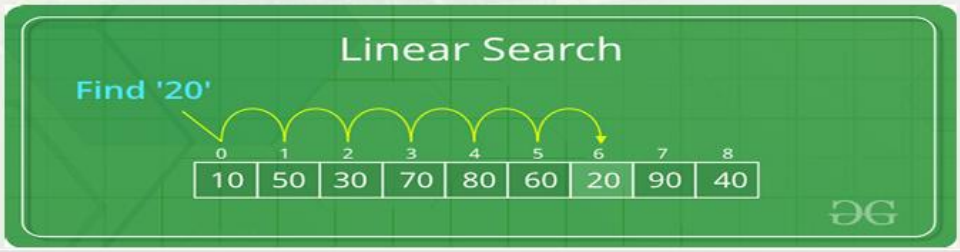
Concepts :

- Algorithm can be applied only on sorted data
- **Mid** = (low + high) / 2 - formula used to find mid
- Given element(Key) compares with middle element of the list.
- If **key=mid** then element is found
- Otherwise list divide into two part.(key <mid) (key >mid) **First to mid-1** or **mid+1 to last**.


Binary Search
Search 23
0 1 2 3 4 5 6 7 8 9
2 5 8 12 16 23 38 56 72 91
23 > 16
take 2nd half
L=0 1 2 3 M=4 5 6 7 8 H=9
2 5 8 12 16 23 38 56 72 91
23 > 56
take 1st half
0 1 2 3 4 L=5 6 M=7 8 H=9
2 5 8 12 16 23 38 56 72 91
Found 23,
Return 5
0 1 2 3 4 L=5,M=5 H=6 7 8 9
2 5 8 12 16 23 38 56 72 91

Linear Search

- Since the array elements are stored in linear order searching the element in the linear order make it easy and efficient.
- The search may be successful or unsuccessfully. That is, if the required element is found them the search is successful other wise it is unsuccessfully.



Algorithm:

1. Linear Search:

- (a) Start from the first element of the list.
- (b) Compare the current element with the target value.
- (c) If it matches, return its index.
- (d) If the end of the list is reached without a match, return -1.

2. Binary Search:

- (a) Sort the list if it is not already sorted.
- (b) Initialise low = 0 and high = n-1.
- (c) While low <= high, compute mid = (low+high)//2.
- (d) If arr[mid] equals the target, return mid.
- (e) If arr[mid] is less than the target, set low = mid+1, otherwise set high = mid-1.
- (f) If the loop terminates without a match, return -1.

Functions used:

1. Linear Search --------

```
def linear_search(customer_list, target_id)
```

2. Binary Search --------

```
def binary_search(sorted_list, target_id)
```

Input:

customer_ids = [102, 205, 150, 123, 301, 110, 175]

Output:

Enter Customer ID to search: 102
Linear Search: Found
Binary Search: Found

Conclusion:

Thus, we have successfully implemented Linear and Binary Search Algorithms.

----------program with output printout ----------

```
# customer_search.py

# Sample list of customer account IDs

customer_ids = [102, 205, 150, 123, 301, 110, 175]

# -------- a) Linear Search --------

def linear_search(customer_list, target_id):

    for i in range(len(customer_list)):

        if customer_list[i] == target_id:

            return True

    return False


# -------- b) Binary Search --------

def binary_search(sorted_list, target_id):

    low = 0

    high = len(sorted_list) - 1


    while low <= high:

        mid = (low + high) // 2
```

```
        if sorted_list[mid] == target_id:
            return True
        elif sorted_list[mid] < target_id:
            low = mid + 1
        else:
            high = mid - 1
    return False

# -------- Testing the Searches --------
search_id = int(input("Enter Customer ID to search: "))

# Linear Search
found_linear = linear_search(customer_ids, search_id)
print("Linear Search: Found" if found_linear else "Linear Search: Not Found")

# Binary Search (requires sorted list)
sorted_ids = sorted(customer_ids)
found_binary = binary_search(sorted_ids, search_id)
print("Binary Search: Found" if found_binary else "Binary Search: Not Found")
```

Practical 3

AIM:

Write a Python program to sort employee salaries stored in a list, in ascending order, using Selection Sort and Bubble Sort, and display the top five highest salaries.

PROBLEM STATEMENT:

In a company, employee salaries are stored in a list as floating-point numbers. Write a Python program that sorts the employee salaries in ascending order using (i) Selection Sort and (ii) Bubble Sort. After sorting, the program should display the top five highest salaries in the company.

OBJECTIVE:

1. To understand the working of comparison-based sorting algorithms.
2. To implement Selection Sort and Bubble Sort in Python.
3. To analyze and compare the time complexity of the two sorting techniques.

OUTCOME:

1. Students will be able to implement Selection Sort and Bubble Sort algorithms.
2. Students will be able to sort real-valued data and extract summary information such as the top-k values.

SOFTWARE / HARDWARE USED:

- Programming Language: Python 3.x
- IDE: VS Code / PyCharm / Google Colab / Jupyter Notebook

THEORY:

1. Sorting:

Sorting is the process of arranging the elements of a list in a particular order — ascending or descending. It is a fundamental operation used to simplify searching, ranking and reporting tasks.

2. Selection Sort:

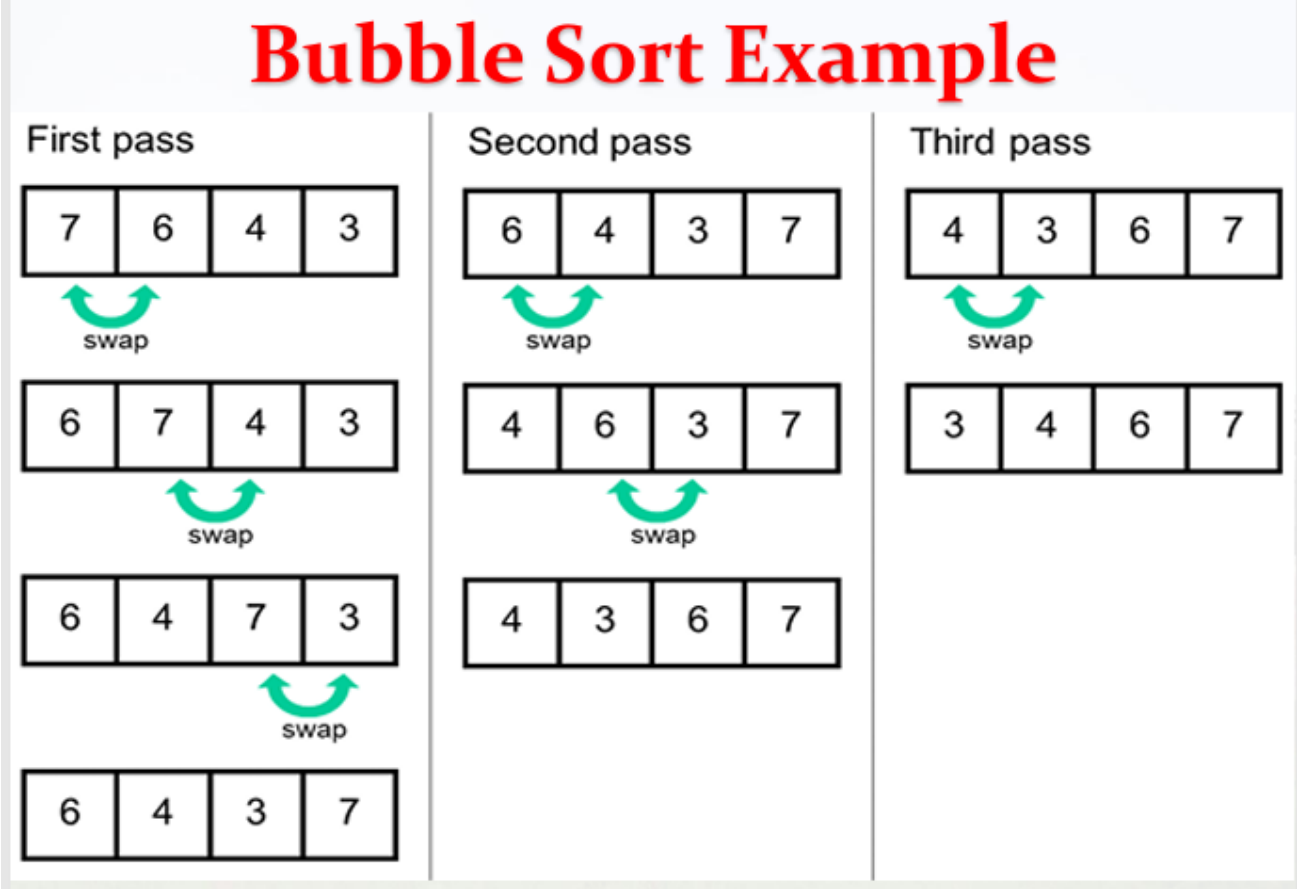
Selection sort repeatedly selects the minimum element from the unsorted part of the list and places it at the beginning of the unsorted part. It performs O(n^2) comparisons but only O(n) swaps, making it useful when the cost of swapping is high.

3. Bubble Sort:

Bubble sort repeatedly steps through the list, compares adjacent elements and swaps them if they are in the wrong order. Larger elements progressively 'bubble' towards the end of the list. It has a worst-case time complexity of O(n^2) but can be optimised to O(n) for an already-sorted list using an early-exit flag.

Algorithm Bubble sorting

1. In Bubble sort pairs of adjacent elements (start from 0^{th} and 1^{st} locations) is compared and then swapping is performed when first element is greater than another element in pair.
2. Repeat step 1 until (n - 2) position element is compared with (n - 1) position element.
3. Here first iteration is completed and largest value in list is stored at (n - 1) location.
4. Now start second iteration, Repeat step 1 until (n - 3) position element is compared with (n - 2) position element.
5. If there are n elements, then (n - 1) passes are required.


Bubble Sort Example
First pass
7 6 4 3
swap
6 7 4 3
swap
6 4 7 3
swap
6 4 3 7
Second pass
6 4 3 7
swap
4 6 3 7
swap
4 3 6 7
Third pass
4 3 6 7
swap
3 4 6 7

Selection Sort

Elements are : 5 , 9, 1, 11, 2, 4

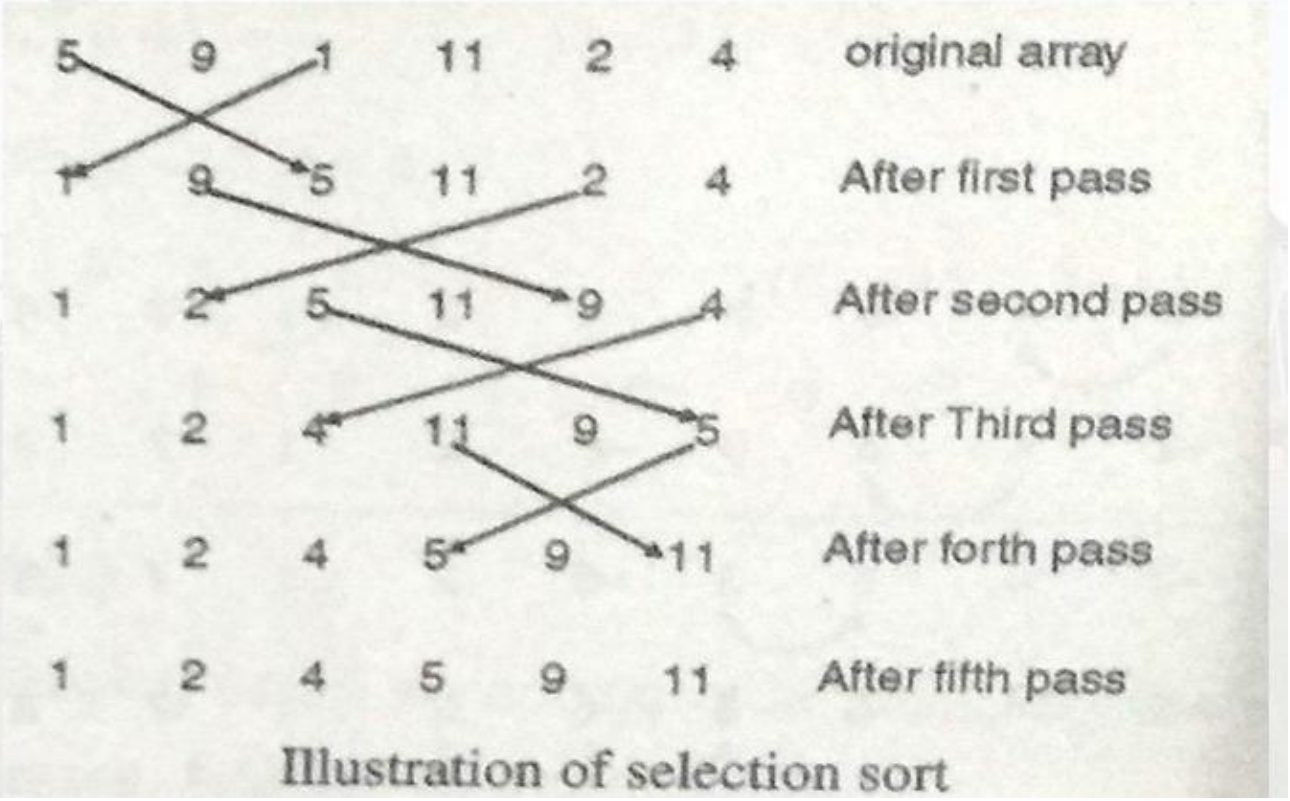
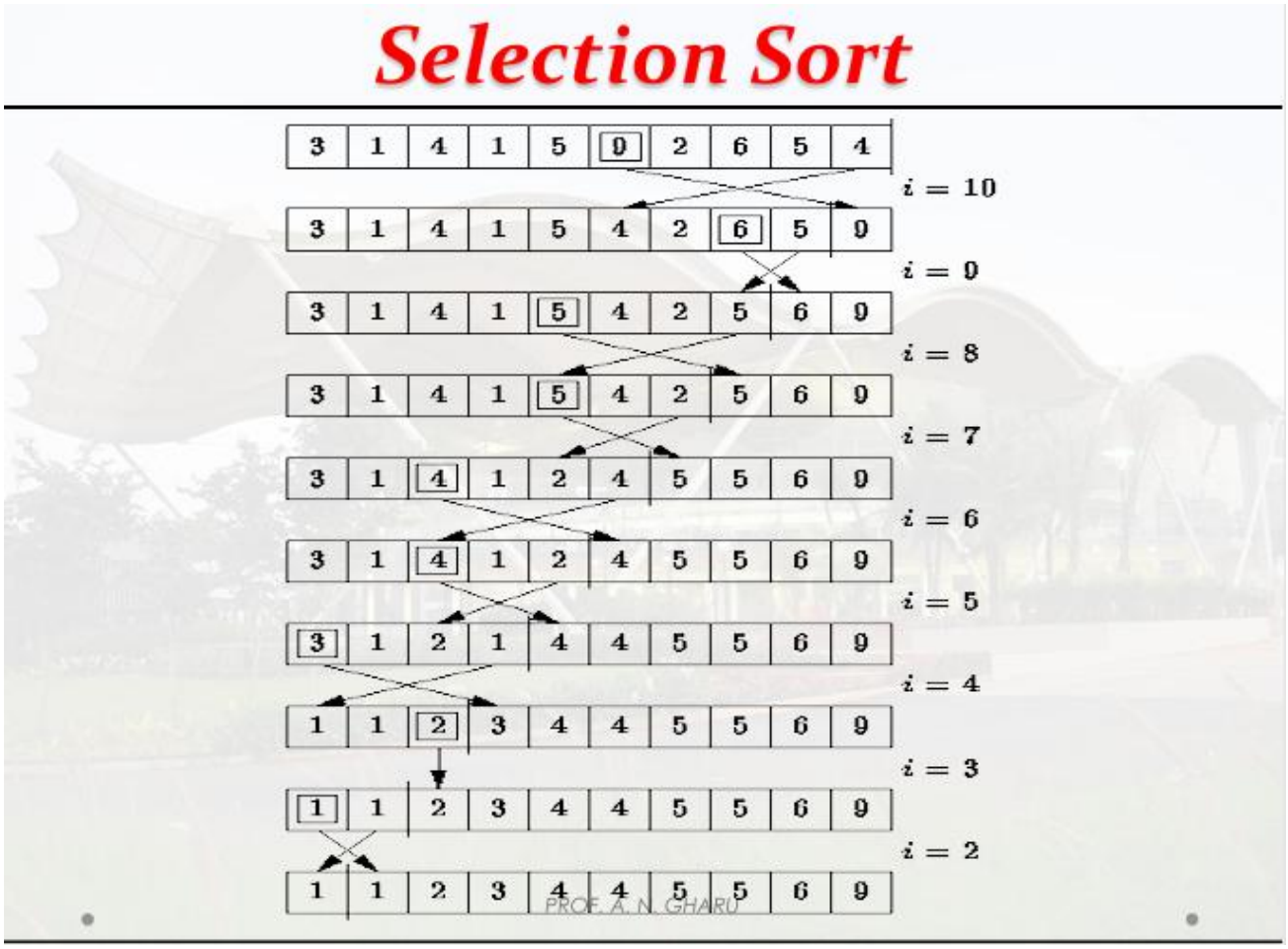


Illustration of selection sort



Algorithm:

1. Selection Sort: (a) For i from 0 to n-2, find the index of the minimum element in arr[i..n-1]. (b) Swap the minimum element with arr[i]. (c) Repeat until the entire list is sorted.
2. Bubble Sort: (a) For each pass i from 0 to n-2, compare every pair of adjacent elements arr[j], arr[j+1]. (b) Swap them if arr[j] > arr[j+1]. (c) If no swaps occur in a pass, the list is already sorted and the algorithm can stop early.
3. After sorting the list using either method, slice the last five elements of the ascending list (or the first five of a descending copy) to obtain the top five highest salaries.

Functions Used:

a) Selection Sort --------

```
def selection_sort(salary_list)
```

b) Bubble Sort --------

```
def bubble_sort(salary_list):
```

Input:

```
# Sample list of employee salaries
salaries = [45000.0, 38000.5, 52000.75, 61000.0, 47000.0, 70000.0, 56000.0, 49000.5]
```

Output:

Top 5 salaries (Selection Sort): [70000.0, 61000.0, 56000.0, 52000.75, 49000.5]

Top 5 salaries (Bubble Sort): [70000.0, 61000.0, 56000.0, 52000.75, 49000.5]

Conclusion:

Thus, we have successfully implemented Bubble and Selection Sort Algorithms.

----------program----------

```
# employee_salary_sort.py

# Sample list of employee salaries

salaries = [45000.0, 38000.5, 52000.75, 61000.0, 47000.0, 70000.0, 56000.0, 49000.5]

# -------- a) Selection Sort --------

def selection_sort(salary_list):

    n = len(salary_list)

    for i in range(n):

        min_index = i

        for j in range(i + 1, n):

            if salary_list[j] < salary_list[min_index]:

                min_index = j

        salary_list[i], salary_list[min_index] = salary_list[min_index], salary_list[i]

    return salary_list

# -------- b) Bubble Sort --------

def bubble_sort(salary_list):

    n = len(salary_list)

    for i in range(n):

        for j in range(0, n - i - 1):

            if salary_list[j] > salary_list[j + 1]:
```

```
            salary_list[j], salary_list[j + 1] = salary_list[j + 1], salary_list[j]
    return salary_list
# -------- Run both sorts and display top 5 salaries --------
# Using Selection Sort
sorted_salaries_selection = selection_sort(salaries.copy())
print("Top 5 salaries (Selection Sort):", sorted_salaries_selection[-5:][::-1])

# Using Bubble Sort
sorted_salaries_bubble = bubble_sort(salaries.copy())
print("Top 5 salaries (Bubble Sort):", sorted_salaries_bubble[-5:][::-1])
```

Practical 4

AIM:

Implement a real-time Undo/Redo system for a text editing application using the Stack data structure.

PROBLEM STATEMENT:

Implement a real-time undo/redo system for a text editing application using a Stack data structure. The system should support the following operations:

Make a Change (a new change is made to the document),

Undo Action (revert the most recent change and store it for potential redo),

Redo Action (reapply the most recently undone action), and

Display Document State (show the current state of the document after undoing or redoing an action).

OBJECTIVE:

1. To understand the LIFO (Last-In-First-Out) principle of the Stack data structure.
2. To implement push and pop operations of a stack in Python.
3. To apply the stack data structure to design a practical undo/redo system.

OUTCOME:

1. Students will be able to implement stack operations (push, pop, peek) in Python.
2. Students will be able to apply stacks to solve real-world application-based problems such as undo/redo functionality.

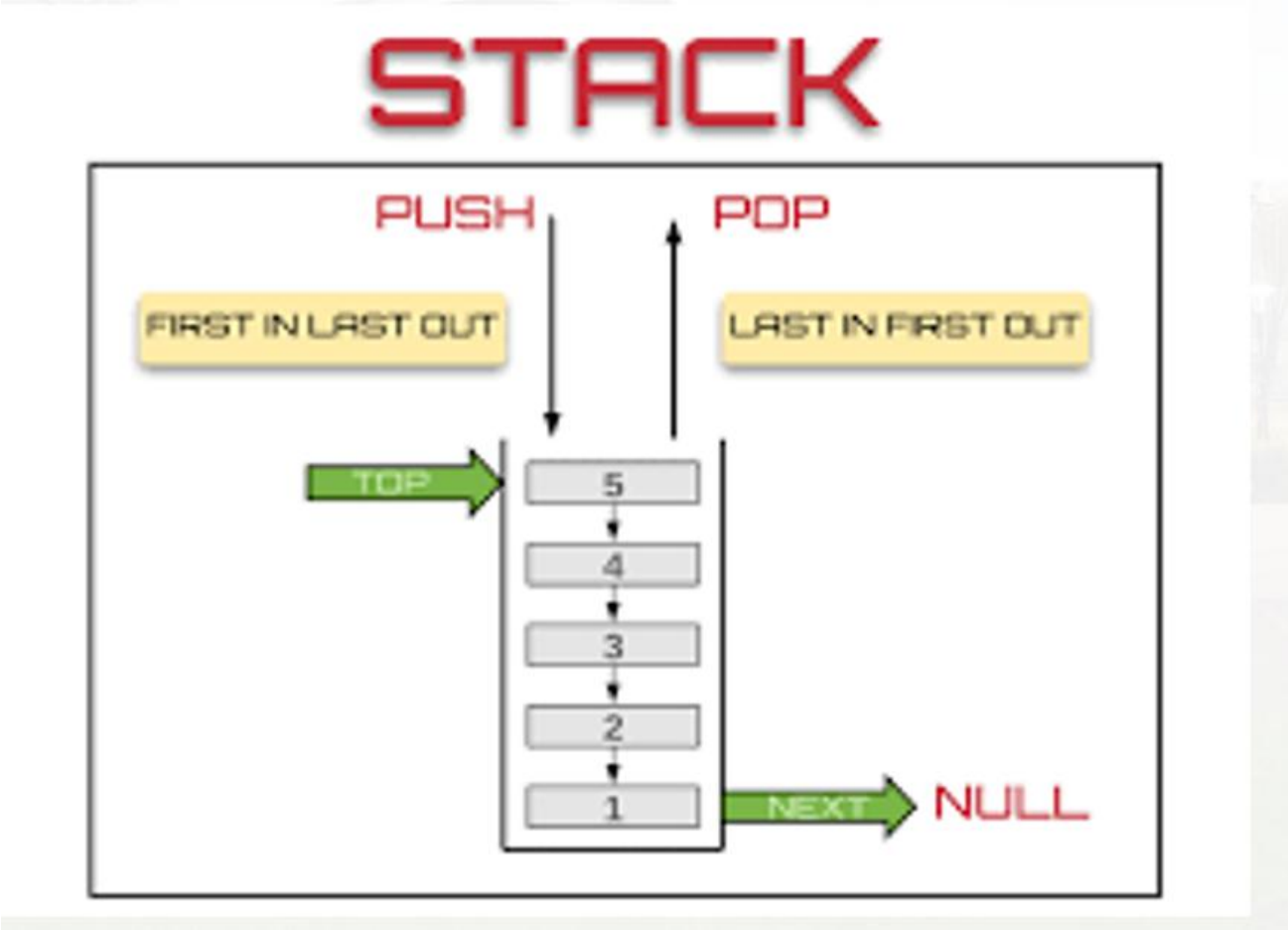
SOFTWARE / HARDWARE USED:

- Programming Language: Python 3.x
- IDE: VS Code / PyCharm / Google Colab / Jupyter Notebook

THEORY:

1. Stack:

A stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle — the last element inserted (pushed) is the first one to be removed (popped). The two primary operations are push (insert at the top) and pop (remove from the top), both of which take O (1) time.



2. Undo/Redo using Two Stacks:

An undo/redo system can be implemented using two stacks: an undo_stack that stores the history of changes made to the document, and a redo_stack that stores changes that have been undone. Making a new change pushes the change onto undo_stack and clears redo_stack (since the redo history becomes invalid). Undo pops the most recent change from undo_stack and pushes it onto redo_stack. Redo pops from redo_stack and pushes it back onto undo_stack.

STACK AS AN ADT

Createstack - it create new empty stack.

push( ) − Pushing (storing) an element on the stack.

pop( ) − Removing an element from the stack.

peek( ) − get the top data element of the stack, without removing it.

isFull( ) − check if stack is full.

isEmpty( ) − check if stack is empty.

Size() – return the number of item in the stack.

Algorithm:

1. Initialise an empty undo_stack, an empty redo_stack, and the document state as an empty string.
2. Make a Change: push the current state onto undo_stack, clear redo_stack, and update the document to the new state.

3. Undo: if undo_stack is not empty, push the current state onto redo_stack, pop the previous state from undo_stack and set it as the current document state.

4. Redo: if redo_stack is not empty, push the current state onto undo_stack, pop the next state from redo_stack and set it as the current document state.
5. Display Document State: print the current value of the document.

Functions Used:

- def make_change(self, change)
- def undo_action(self)
- def redo_action(self)
- def display_state(self)

Conclusion: Thus, we have successfully implemented Redo Undo Stack

----------PROFRAM----------

```
class TextEditor:

  def __init__(self):

     self.document = ""

     self.undo_stack = []

     self.redo_stack = []


  def make_change(self, change):

     self.undo_stack.append(self.document)

     self.document += change

     self.redo_stack.clear()

     print("\nChange made.")

     self.display_state()


  def undo_action(self):

     if self.undo_stack:

        self.redo_stack.append(self.document)
```

```
            self.document = self.undo_stack.pop()
            print("\nUndo performed.")
        else:
            print("\nNo more actions to undo.")
        self.display_state()

    def redo_action(self):
        if self.redo_stack:
            self.undo_stack.append(self.document)
            self.document = self.redo_stack.pop()
            print("\nRedo performed.")
        else:
            print("\nNo more actions to redo.")
        self.display_state()

    def display_state(self):
        print("Current Document State: '" + self.document + "'")

    def run_editor(self):
        while True:
            print("\n--- MENU ---")
            print("1. Make a Change")
            print("2. Undo")
            print("3. Redo")
            print("4. Display Document State")
            print("5. Exit")
```

```
        choice = input("Enter your choice: ")

        if choice == '1':
            change = input("Enter text to add: ")
            self.make_change(change)
        elif choice == '2':
            self.undo_action()
        elif choice == '3':
            self.redo_action()
        elif choice == '4':
            self.display_state()
        elif choice == '5':
            print("Exiting...")
            break
        else:
            print("Invalid choice. Try again.")

# Run the editor
editor = TextEditor()
editor.run_editor()
```

Practical 5

AIM:

Simulate the call queue of a call center using the Queue data structure.

PROBLEM STATEMENT:

A call center receives incoming calls, and each call is assigned a unique customer ID. Calls are answered in the order in which they are received. Simulate the call queue of a call center using a queue data structure that supports: addCall(customerID, callTime) — add a call to the queue; answerCall() — answer and remove the first call from the queue; viewQueue() — view all calls currently in the queue without removing them; isQueueEmpty() — check if the queue is empty.

OBJECTIVE:

1. To understand the FIFO (First-In-First-Out) principle of the Queue data structure.
2. To implement enqueue and dequeue operations of a queue in Python.
3. To apply the queue data structure to simulate a real-world call-handling system.

OUTCOME:

1. Students will be able to implement queue operations (enqueue, dequeue, peek) in Python.
2. Students will be able to apply queues to model real-world, order-preserving systems such as a call center.

SOFTWARE / HARDWARE USED:

- Programming Language: Python 3.x
- IDE: VS Code / PyCharm / Google Colab / Jupyter Notebook

THEORY:

1. Queue:

A queue is a linear data structure that follows the First-In-First-Out (FIFO) principle — the first element inserted is the first one to be removed. The two primary operations are enqueue (insert at the rear) and dequeue (remove from the front).

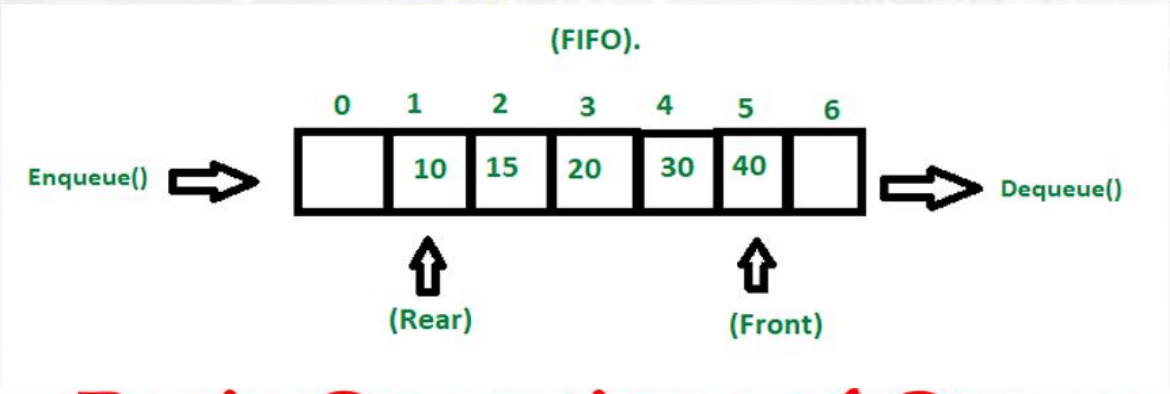
2. collections.deque:

Python's collections.deque provides an efficient, O(1) implementation of both enqueue (append) and dequeue (popleft) operations, unlike a plain list where removing from the front takes O(n) time because every remaining element has to be shifted.

FIFO Principle of Queue:

A Queue is like a line waiting to purchase tickets, where the first person in line is the first person served. (i.e. First come first serve).

Position of the entry in a queue ready to be served, that is, the first entry that will be removed from the queue, is called the **front** of the queue(sometimes, **head** of the queue), similarly, the position of the last entry in the queue, that is, the one most recently added, is called the **rear** (or the **tail**) of the queue. See the below figure.



Basic Operations of Queue

Queue operations work as follows:

- Two pointers FRONT and REAR
- FRONT track the first element of the queue
- REAR track the last element of the queue
- initially, set value of FRONT and REAR to -1

Algorithm:

1. Initialise an empty queue using collections.deque.
2. addCall(customerID, callTime): append a tuple (customerID, callTime) to the rear of the queue — O(1).
3. answerCall(): if the queue is not empty, remove and return the call at the front of the queue using popleft() — O(1); otherwise report that the queue is empty.
4. viewQueue(): iterate over and print the queue without modifying it — O(n).
5. isQueueEmpty(): return True if the length of the queue is zero, else False — O(1).

Functions Used:

- # Add a call to the queue

  def addCall(customerID, callTime):

- # Answer the first call in the queue

  def answerCall():

- # View all calls currently in the queue

  def viewQueue():

- # Check if the queue is empty

  def isQueueEmpty():

Conclusion:

Thus, We have successfully implemented real event processing system using Queue Data Structure

----------program----------

```
# Call Center Queue Simulation using a Queue (FIFO)


# Define the call queue as a list

call_queue = []


# Add a call to the queue

def addCall(customerID, callTime):

    call = {"CustomerID": customerID, "CallTime": callTime}

    call_queue.append(call)

    print(f"Call from Customer {customerID} added (Call Time: {callTime} mins)")
```

```
# Answer the first call in the queue
def answerCall():
    if call_queue:
        call = call_queue.pop(0)
        print(f"Answering call from Customer {call['CustomerID']} (Call Time: {call['CallTime']} mins)")
    else:
        print("No calls to answer.")

# View all calls currently in the queue
def viewQueue():
    if call_queue:
        print("Current Call Queue:")
        for i, call in enumerate(call_queue, 1):
            print(f"{i}. Customer {call['CustomerID']} - {call['CallTime']} mins")
    else:
        print("Call queue is empty.")

# Check if the queue is empty
def isQueueEmpty():
    if not call_queue:
        print("The call queue is empty.")
    else:
        print("The call queue is not empty.")

# Menu to interact with the system
def menu():
```

```
while True:
    print("\n--- CALL CENTER MENU ---")
    print("1. Add Call")
    print("2. Answer Call")
    print("3. View Call Queue")
    print("4. Check if Queue is Empty")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == '1':
        customerID = input("Enter Customer ID: ")
        callTime = input("Enter Call Time (in minutes): ")
        addCall(customerID, callTime)
    elif choice == '2':
        answerCall()
    elif choice == '3':
        viewQueue()
    elif choice == '4':
        isQueueEmpty()
    elif choice == '5':
        print("Exiting Call Center System.")
        break
    else:
        print("Invalid choice. Please enter a number from 1 to 5.")
```

```
# Run the program

menu()
```

Practical 6

AIM:

Create a Student Record Management System using a linked list to store student data (Roll No, Name, Marks) and perform Add, Delete, Update, Search and Sort operations.

PROBLEM STATEMENT:

A college wants to maintain student records consisting of Roll No, Name and Marks. Design and implement a Student Record Management System using a doubly linked list to store the student records. The system should support the following operations: (i) Add a new student record, (ii) Delete a student record, (iii) Update an existing student record, (iv) Search for a student record by roll number, and (v) Sort and display the records in ascending or descending order based on marks or roll number.

OBJECTIVE:

1. To understand the structure and working of a doubly linked list.
2. To implement insertion, deletion, searching and updating of nodes in a doubly linked list.
3. To implement sorting of linked list data and displaying records in ascending/descending order.

OUTCOME:

1. Students will be able to design a Node class and build a doubly linked list to store multi-field records.
2. Students will be able to implement Add, Delete, Update, Search and Sort operations on a linked list and apply them to a real-world record-management problem.

SOFTWARE / HARDWARE USED:

- Programming Language: Python 3.x
- IDE: VS Code / PyCharm / Google Colab / Jupyter Notebook

THEORY:

1. Linked List:

A linked list is a linear data structure in which each element (called a node) is stored at a separate memory location and is linked to the next element using a pointer/reference, instead of being stored in contiguous memory locations like an array. This allows efficient insertion and deletion without shifting other elements.

2. Doubly Linked List:

In a doubly linked list, every node stores a reference to both the next node and the previous node, in addition to its data. This allows traversal in both the forward and backward directions and makes deletion of a given node easier since the previous node does not need to be searched for separately.

3. Node Structure for Student Records:

Each node of the linked list in this experiment stores three data fields — Roll No, Name and Marks — along with the prev and next pointers, so that a single node fully represents one student's record.

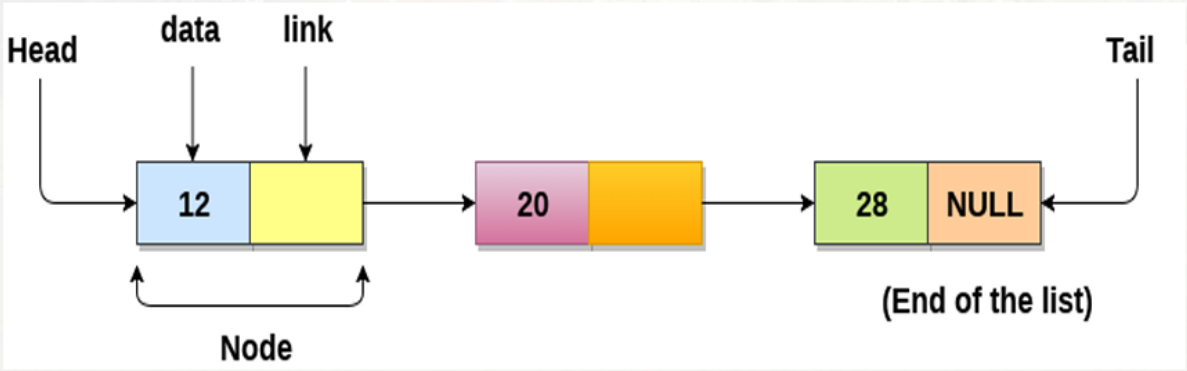
4. Sorting a Linked List:

Since a linked list does not support direct indexing like an array, sorting is done by traversing the list and comparing the data of adjacent nodes, swapping their data values (Roll No, Name, Marks) whenever they

are out of the required order. Repeating this pass until no more swaps are needed (a bubble-sort approach) sorts the entire list in ascending or descending order, based on the selected key (marks or roll number).

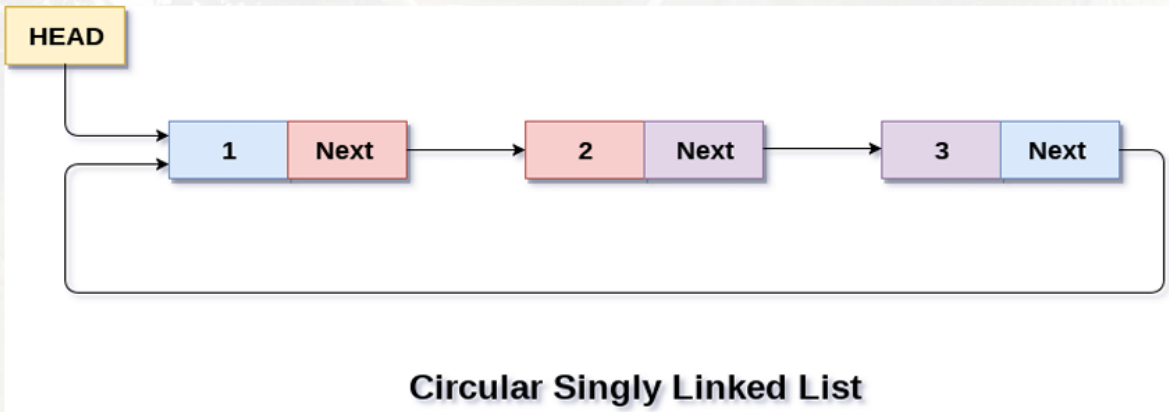
Linked List can be defined as collection of objects called nodes that are randomly stored in the memory.

- A node contains two fields i.e. data stored at that particular address and the pointer which contains the address of the next node in the memory.
- The last node of the list contains pointer to the null.



SINGLY CIRCULAR LINKED LIST

"Circular Linked List is a variation of Linked list in which the first element points to the last element and the last element points to the first element. Both Singly Linked List and Doubly Linked List can be made into a circular linked list".



Circular Singly Linked List

Algorithm:

1. Define a Node class with fields roll, name, marks, and pointers prev and next.
2. Add(roll, name, marks): create a new node and attach it after the current tail node of the list; update the tail pointer.
3. Search(roll): traverse the list from the head node, comparing the roll number of each node with the target, until a match is found or the list ends.
4. Update(roll, ...): use Search to locate the node with the given roll number, then modify its name and/or marks fields.
5. Delete(roll): use Search to locate the node; adjust the next pointer of its previous node and the prev pointer of its next node to remove it from the list.
6. Sort(key, order): repeatedly scan the list, comparing the chosen key (marks or roll) of adjacent nodes, and swap their data values if they are not in the required ascending/descending order; repeat until the entire list is sorted.
7. Display(): traverse the list from the head node to the end, printing the Roll No, Name and Marks of every record.

----------program----------

```
# Student Record Management System using Singly Linked List

class StudentNode:
    def __init__(self, roll_no, name, marks):
        self.roll_no = roll_no
        self.name = name
        self.marks = marks
        self.next = None

class StudentLinkedList:
    def __init__(self):
        self.head = None

    def add_student(self, roll_no, name, marks):
        new_node = StudentNode(roll_no, name, marks)
        if self.head is None:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node
        print("□ Student record added.")

    def delete_student(self, roll_no):
        current = self.head
        prev = None
        while current:
```

```
            if current.roll_no == roll_no:
                if prev:
                    prev.next = current.next
                else:
                    self.head = current.next
                print("□ Student record deleted.")
                return
            prev = current
            current = current.next
        print("□□ Student not found.")

    def update_student(self, roll_no, new_name, new_marks):
        current = self.head
        while current:
            if current.roll_no == roll_no:
                current.name = new_name
                current.marks = new_marks
                print("□ Student record updated.")
                return
            current = current.next
        print("□□ Student not found.")

    def search_student(self, roll_no):
        current = self.head
        while current:
            if current.roll_no == roll_no:
                print(f"□ Found: Roll No: {current.roll_no}, Name:
{current.name}, Marks: {current.marks}")
                return
            current = current.next
        print(" Student not found.")

    def display_students(self, sort_by="roll_no", ascending=True):
        students = []
        current = self.head
        while current:
            students.append((current.roll_no, current.name, current.marks))
            current = current.next

        if sort_by == "roll_no":
            students.sort(key=lambda x: x[0], reverse=not ascending)
        elif sort_by == "marks":
            students.sort(key=lambda x: x[2], reverse=not ascending)

        if not students:
            print("□□ No records to display.")
            return
```

```
        print("\n Student Records:")
        for s in students:
            print(f"Roll No: {s[0]}, Name: {s[1]}, Marks: {s[2]}")

# Menu for interaction
def menu():
    system = StudentLinkedList()
    while True:
        print("\n--- Student Record Management Menu ---")
        print("1. Add Student")
        print("2. Delete Student")
        print("3. Update Student")
        print("4. Search Student")
        print("5. Display All Students")
        print("6. Exit")

        choice = input("Enter your choice (1-6): ")

        if choice == '1':
            roll = int(input("Enter Roll No: "))
            name = input("Enter Name: ")
            marks = int(input("Enter Marks: "))
            system.add_student(roll, name, marks)
        elif choice == '2':
            roll = int(input("Enter Roll No to Delete: "))
            system.delete_student(roll)
        elif choice == '3':
            roll = int(input("Enter Roll No to Update: "))
            name = input("Enter New Name: ")
            marks = int(input("Enter New Marks: "))
            system.update_student(roll, name, marks)
        elif choice == '4':
            roll = int(input("Enter Roll No to Search: "))
            system.search_student(roll)
        elif choice == '5':
            sort_key = input("Sort by 'roll_no' or 'marks': ").strip()
            order = input("Order 'asc' or 'desc': ").strip()
            ascending = True if order == 'asc' else False
            system.display_students(sort_by=sort_key, ascending=ascending)
        elif choice == '6':
            print(" Exiting Student Record Management System.")
            break
        else:
            print(" Invalid choice. Try again.")

# Start the system
menu()
```

Practical 7

AIM:

Implement a hash table of size 10 using the division method as the hash function, and resolve collisions using chaining.

PROBLEM STATEMENT:

Implement a hash table of size 10 and use the division method as a hash function. In case of a collision, use chaining. Implement the following operations: Insert(key) — insert key-value pairs into the hash table; Search(key) — search for the value associated with a given key; Delete(key) — delete a key-value pair from the hash table.

OBJECTIVE:

1. To understand the concept of hashing and the division method of computing a hash function.
2. To implement collision resolution using chaining (separate linked lists / lists).
3. To implement and analyze insert, search and delete operations on a hash table.

OUTCOME:

1. Students will be able to design a hash table and compute hash values using the division method.
2. Students will be able to implement chaining to resolve collisions and perform insert, search and delete operations.

SOFTWARE / HARDWARE USED:

- Programming Language: Python 3.x
- IDE: VS Code / PyCharm / Google Colab / Jupyter Notebook

THEORY:

1. Hashing:

Hashing is a technique that maps a key to an index of a fixed-size array (the hash table) using a hash function, allowing average-case O(1) insertion, search and deletion.

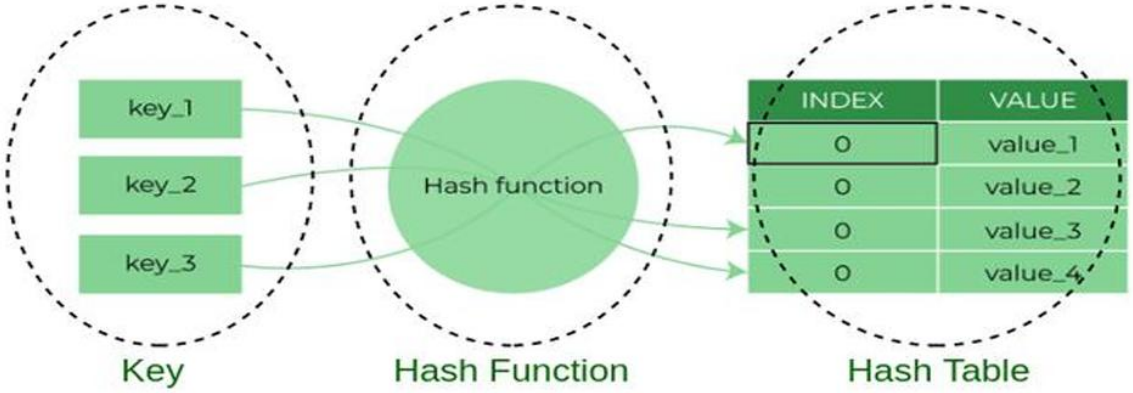
2. Division Method:

The division method computes the hash value as h(key) = key mod table_size. It is simple and fast, and works best when the table size is chosen to be a prime number to reduce clustering.

3. Chaining:

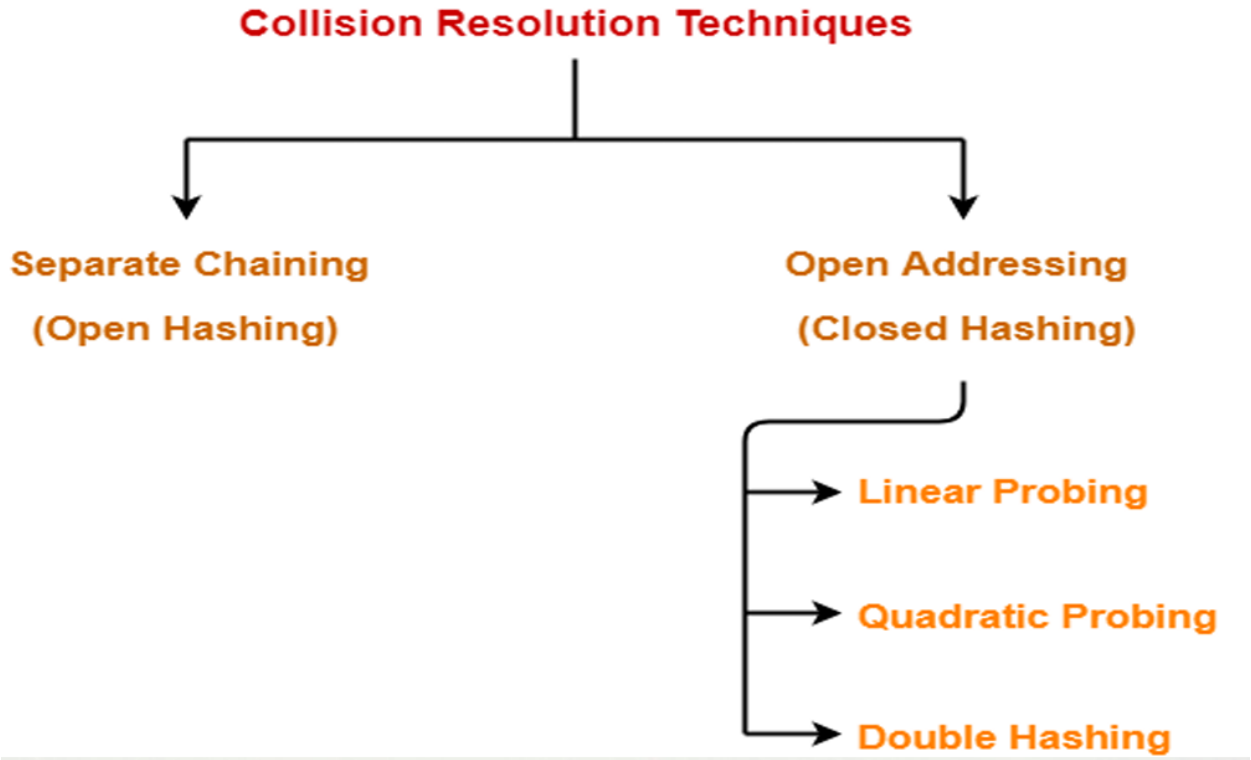
Chaining resolves collisions — the case where two keys hash to the same index — by storing all the keys that hash to the same index in a linked list (or, in Python, a list) attached to that index. Insert, search and delete simply operate on the list at the computed index.

4. Hash Table: A hash table is a data structure that stores records in an array, called a hash table. A Hash table can be used for quick insertion and searching.



Components of Hashing

In Hashing, collision resolution techniques are classified as-



Algorithm:

1. Create an array (list of lists) of size 10, where each slot holds a bucket that stores (key, value) pairs.

2. Insert(key, value): compute index = key % 10; append (key, value) to the bucket at that index; if the key already exists in the bucket, update its value.
3. Search(key): compute index = key % 10; linearly scan the bucket at that index for the key and return its value if found, else report 'not found'.
4. Delete(key): compute index = key % 10; linearly scan the bucket at that index and remove the (key, value) pair if the key is found.

Function used:

- def _hash_function(self, key):
- def insert(self, key, value):
- def search(self, key):

Conclusion:

Thus, We have successfully implemented hash table with collision handling techniques(chaining).

----------program----------

```
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]  # List of empty lists (chaining)

    def _hash_function(self, key):
        return key % self.size

    def insert(self, key, value):
        index = self._hash_function(key)
        # Check if key already exists and update
        for pair in self.table[index]:
            if pair[0] == key:
                pair[1] = value
                print(f"Updated key {key} with value {value}")
```

```
            return
        # Else insert new key-value pair
        self.table[index].append([key, value])
        print(f"Inserted key {key} with value {value}")

    def search(self, key):
        index = self._hash_function(key)
        for pair in self.table[index]:
            if pair[0] == key:
                return pair[1]
        return None  # Not found

    def delete(self, key):
        index = self._hash_function(key)
        for i, pair in enumerate(self.table[index]):
            if pair[0] == key:
                del self.table[index][i]
                print(f"Deleted key {key}")
                return
        print(f"Key {key} not found for deletion.")

    def display(self):
        print("Hash Table:")
        for i, bucket in enumerate(self.table):
            print(f"Index {i}: {bucket}")
```

```
ht = HashTable()


ht.insert(15, "apple")
ht.insert(25, "banana")  # Collision with 15
ht.insert(35, "cherry")  # Collision again


print("Search 25:", ht.search(25))  # Output: banana
ht.delete(25)
print("Search 25 after deletion:", ht.search(25))  # Output: None


ht.display()
```

Practical 8

AIM:

Design and implement a hash table of fixed size using the division method for the hash function and resolve collisions using linear probing.

PROBLEM STATEMENT:

Design and implement a hash table of fixed size. Use the division method for the hash function and resolve collisions using linear probing. Allow the user to perform the following operations: Insert a key, Search for a key, Delete a key, and Display the table.

OBJECTIVE:

1. To understand open addressing as a technique for resolving hash collisions.
2. To implement linear probing for collision resolution in a fixed-size hash table.
3. To implement insert, search, delete and display operations using linear probing, including correct handling of deleted slots.

OUTCOME:

1. Students will be able to implement a hash table using open addressing with linear probing.
2. Students will be able to explain the effect of clustering on the performance of linear probing.

SOFTWARE / HARDWARE USED:

- Programming Language: Python 3.x
- IDE: VS Code / PyCharm / Google Colab / Jupyter Notebook

THEORY:

1. Open Addressing:

In open addressing, all keys are stored directly within the hash table array itself (one key per slot). When a collision occurs, the algorithm probes for the next available slot according to a fixed sequence, instead of using an auxiliary list as in chaining.

2. Linear Probing:

Linear probing resolves a collision at index i by checking the next slot (i+1) mod size, then (i+2) mod size, and so on, until an empty slot is found. It is simple to implement but can suffer from primary clustering, where long runs of occupied slots form and degrade performance.

3. Deletion under Linear Probing:

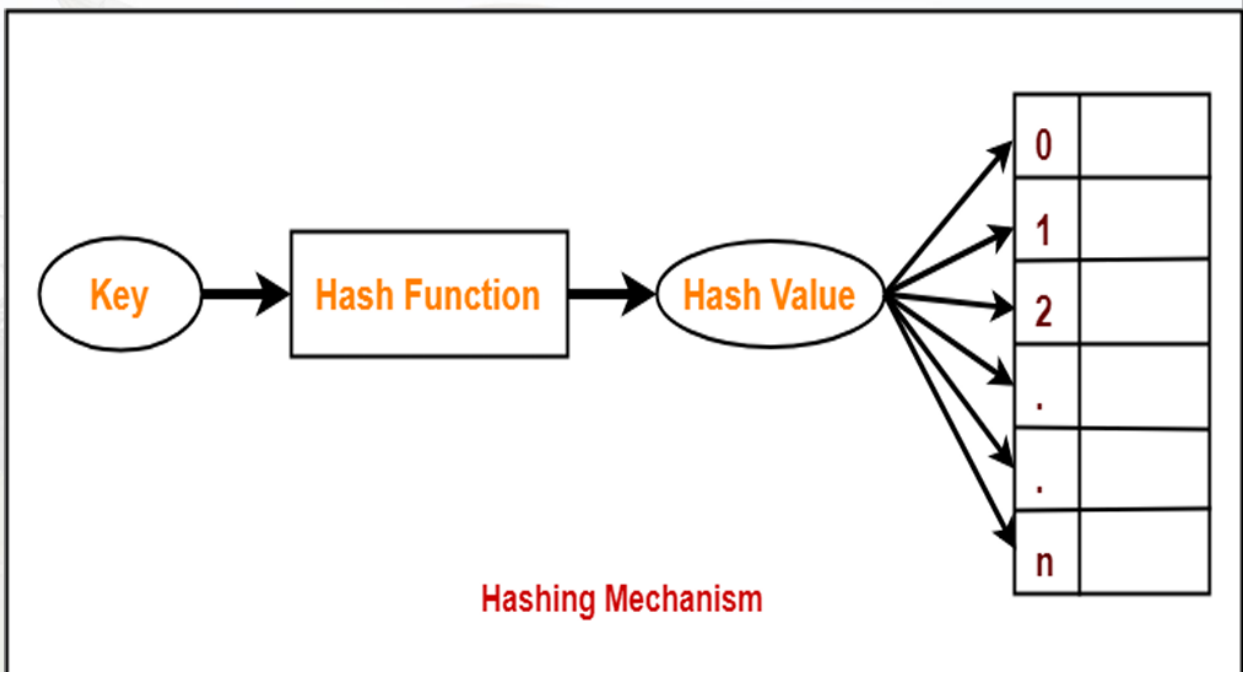
Deleting a key by simply emptying its slot can break the probe chain for later searches. This is handled by marking deleted slots with a special 'DELETED' marker (a tombstone) that search treats as occupied (continue probing) but insert treats as free (available for reuse).

1. **Hashing** is finding an address where the data is to be stored as well as located using a key with the help of the algorithmic function.

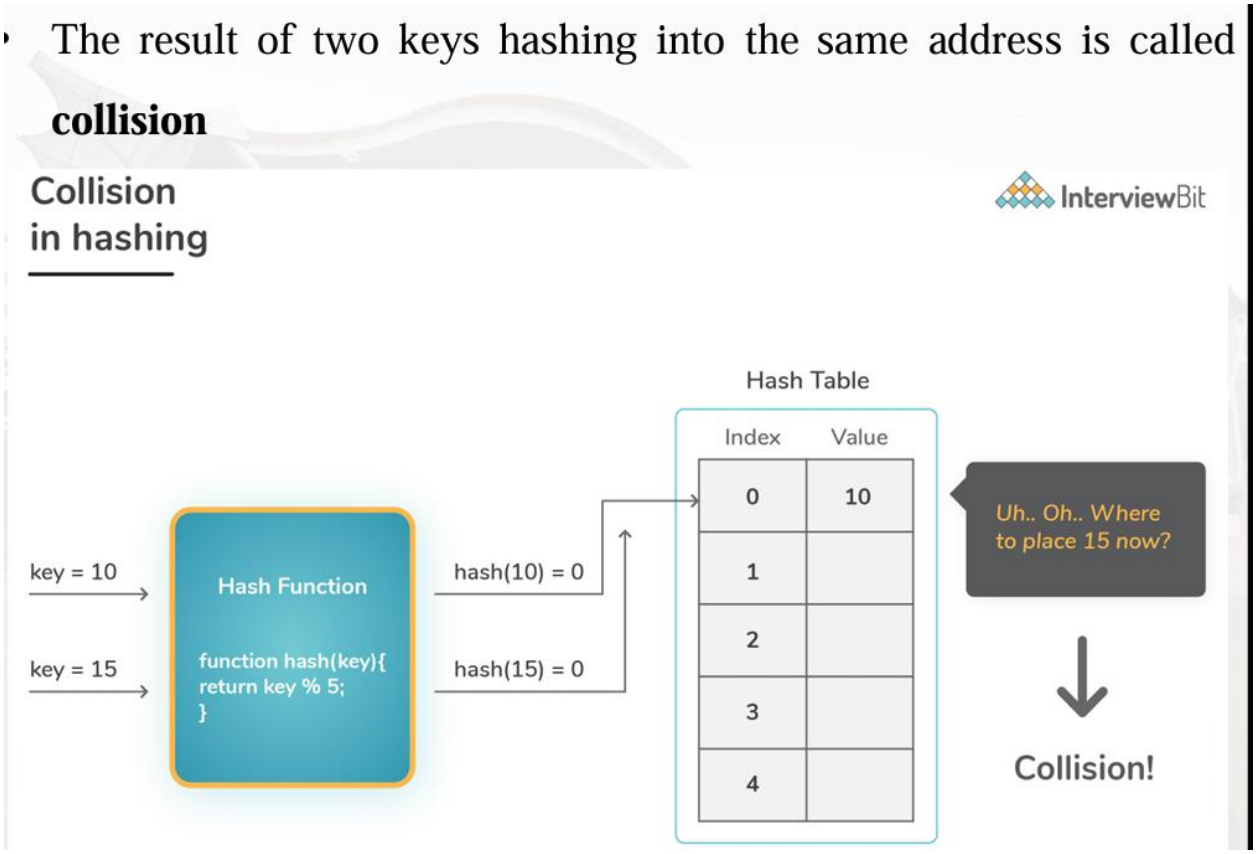
2. **Hashing** is a method of directly computing the address of the record with the help of a key by using a suitable mathematical function called the hash function

3. **A hash table** is an array-based structure used to store <key, information> pairs

- **Hashing Mechanism :**



Hashing Mechanism



Algorithm:

1. Create an array table of fixed size, with every slot initially marked EMPTY.
2. Insert(key): compute index = key % size; probe (index, index+1, index+2, ...) mod size until an EMPTY or DELETED slot is found, then place the key there; if the table is full, report an overflow.
3. Search(key): compute index = key % size; probe successive slots until the key is found (return its slot) or an EMPTY slot is reached (key not present).
4. Delete(key): locate the key using the search procedure and, if found, replace it with a DELETED marker so that later searches can still probe past it.
5. Display(): print the contents of every slot of the table.

Functions used:

- def _hash_function(self, key):
- def insert(self, key):
- def search(self, key):
- def delete(self, key):

Conclusion:

Thus, We have successfully implemented hash table using division method with Linear

probing techniques.

----------program----------

```
class LinearProbingHashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [None] * size
        self.DELETED = "<DELETED>"

    def _hash_function(self, key):
        return key % self.size

    def insert(self, key):
        index = self._hash_function(key)
        original_index = index
        while self.table[index] not in (None, self.DELETED):
            if self.table[index] == key:
                print(f"Key {key} already exists at index {index}.")
                return
            index = (index + 1) % self.size
            if index == original_index:
                print("Hash table is full. Cannot insert.")
                return
        self.table[index] = key
        print(f"Inserted key {key} at index {index}.")

    def search(self, key):
```

```
        index = self._hash_function(key)
        original_index = index
        while self.table[index] is not None:
            if self.table[index] == key:
                print(f"Key {key} found at index {index}.")
                return index
            index = (index + 1) % self.size
            if index == original_index:
                break
        print(f"Key {key} not found.")
        return None

    def delete(self, key):
        index = self.search(key)
        if index is not None:
            self.table[index] = self.DELETED
            print(f"Key {key} deleted from index {index}.")

    def display(self):
        print("Hash Table:")
        for i, key in enumerate(self.table):
            print(f"Index {i}: {key}")
ht = LinearProbingHashTable()

ht.insert(10)
ht.insert(20)
```

```
ht.insert(30)  # Should go to different slots

ht.insert(20)  # Duplicate

ht.display()

ht.search(20)

ht.search(99)


ht.delete(20)

ht.search(20)


ht.display()
```

Practical 9

AIM:

Represent a given graph of locations in an area using an adjacency matrix and an adjacency list, and traverse it using Depth First Search (DFS) and Breadth First Search (BFS) respectively.

PROBLEM STATEMENT:

Consider a particular area in your city. Note the popular locations A, B, C, ... in that area. Assume these locations represent nodes of a graph. If there is a route between two locations, it is represented as a connection (edge) between the corresponding nodes. Find the sequence in which the locations will be visited, starting from a given location (say A), using (i) BFS and (ii) DFS. Represent the graph using an adjacency matrix to perform DFS and an adjacency list to perform BFS.

OBJECTIVE:

1. To understand graph representation techniques — adjacency matrix and adjacency list.
2. To implement Depth First Search (DFS) and Breadth First Search (BFS) graph traversal algorithms.
3. To apply graph traversal to a real-world scenario of visiting locations in a city area.

OUTCOME:

1. Students will be able to represent a graph using both an adjacency matrix and an adjacency list.
2. Students will be able to implement and trace BFS and DFS traversals on a graph.

SOFTWARE / HARDWARE USED:

- Programming Language: Python 3.x
- IDE: VS Code / PyCharm / Google Colab / Jupyter Notebook

THEORY:

1. Graph Representation:

An adjacency matrix is a V x V 2-D array in which cell [i][j] is 1 if an edge exists between vertex i and vertex j, and 0 otherwise; it uses O(V^2) space but allows O(1) edge lookup. An adjacency list stores, for every vertex, a list of the vertices it is directly connected to; it uses O(V+E) space, which is more efficient for sparse graphs (few edges).

2. Depth First Search (DFS):

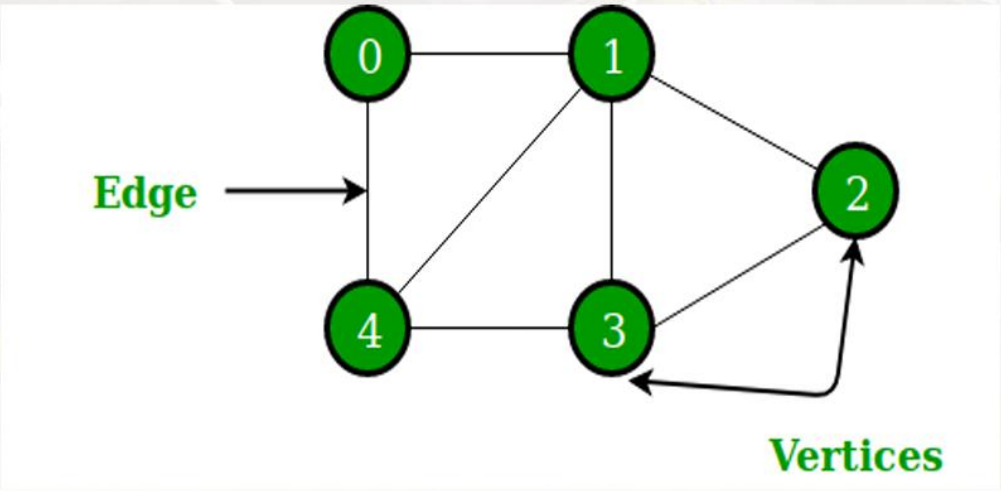
DFS explores as far as possible along each branch before backtracking. It can be implemented recursively, or iteratively using an explicit stack, and visits vertices in a 'go deep first' order.

3. Breadth First Search (BFS):

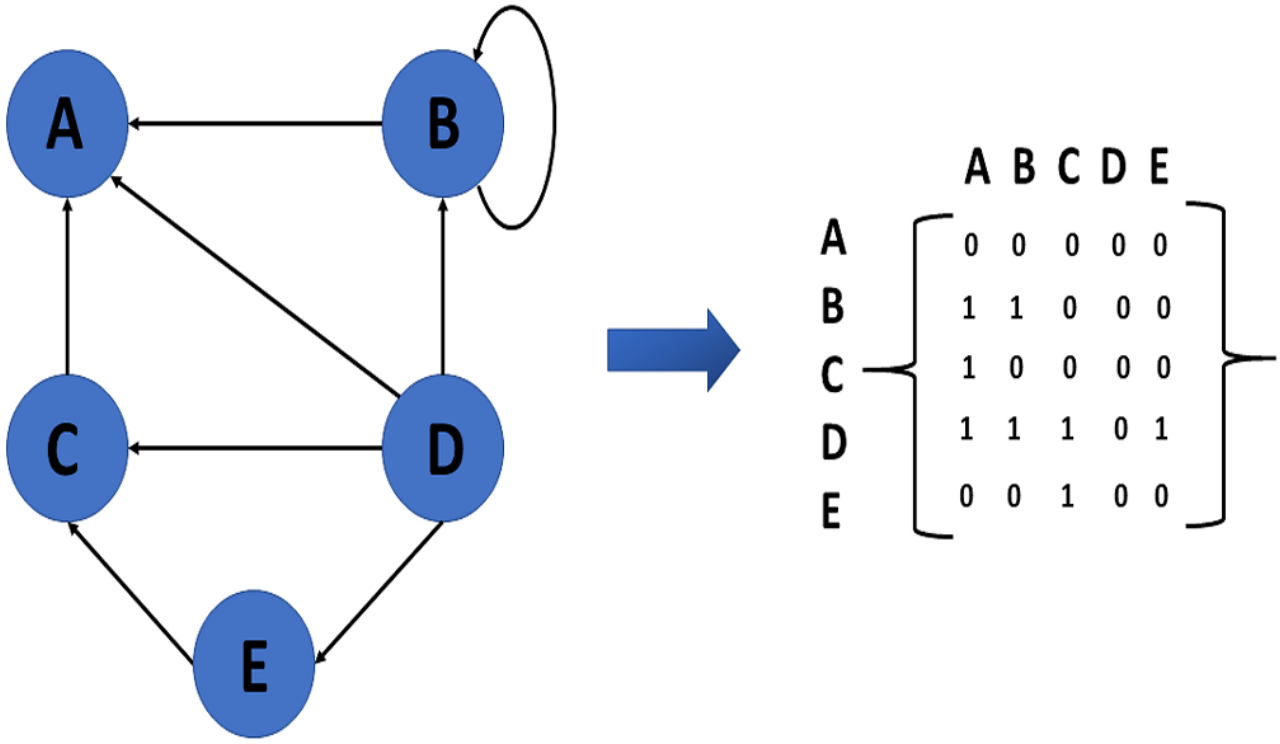
BFS explores all the neighbours of a vertex before moving to the next level of neighbours. It is implemented using a queue and visits vertices level by level, which also makes it useful for finding the shortest path in an unweighted graph.

Introduction of Graph

A graph is a non-linear data structure, which consists of vertices(or nodes) connected by edges(or arcs) where edges may be directed or undirected.

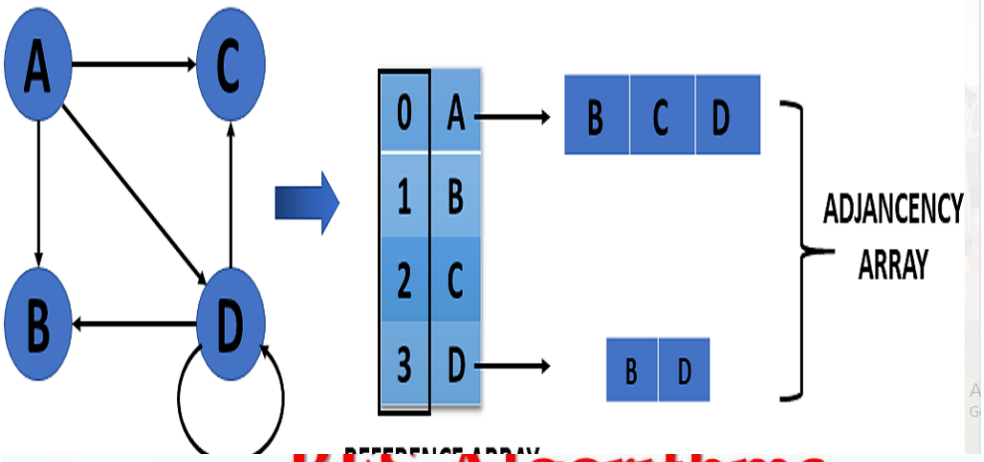


Directed Graph Representation :



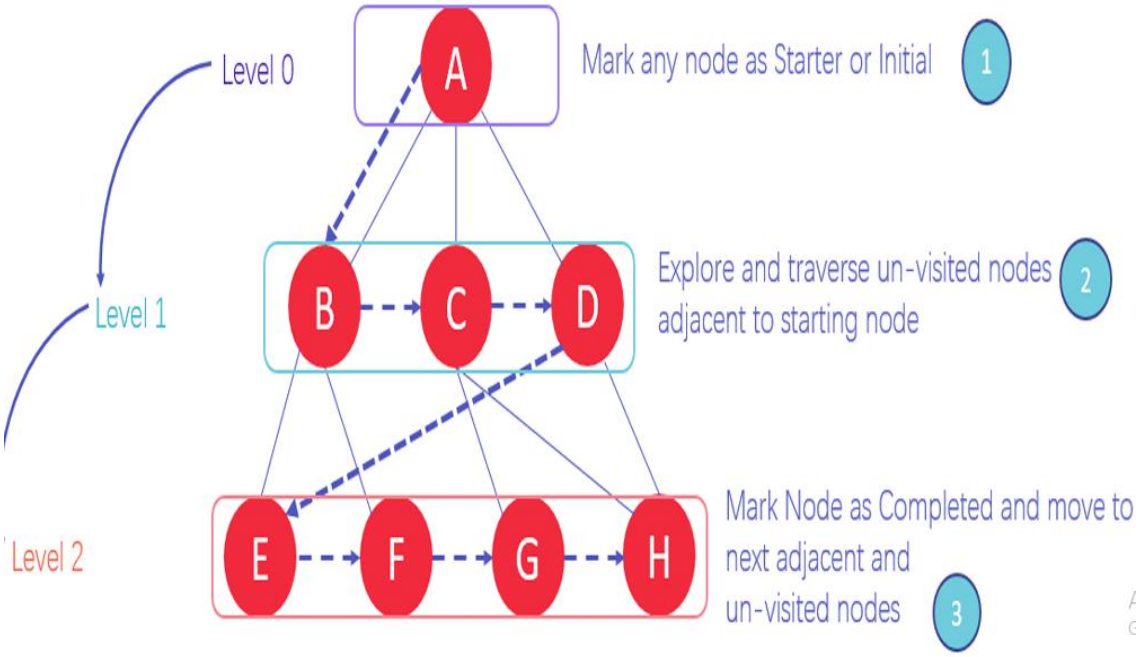
Adjacency List

Weighted Undirected Graph Representation Using an Array



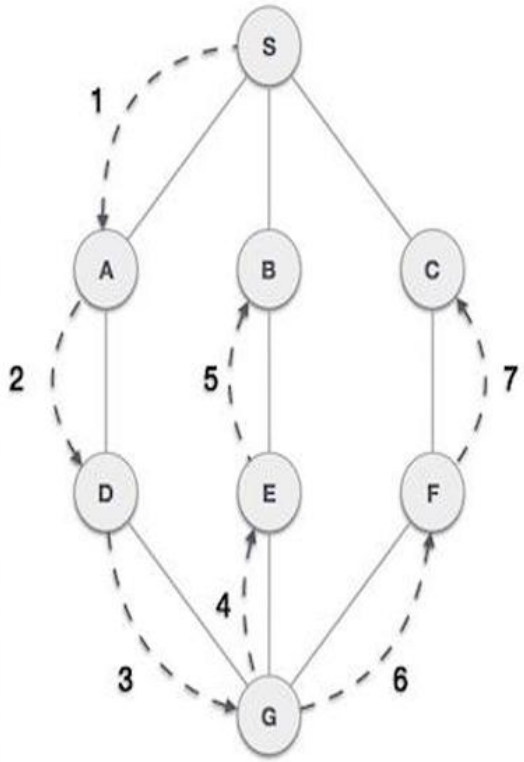
BFS Algorithms

CONCEPT DIAGRAM



DFS Algorithms

Depth First Search (DFS) algorithm traverses a graph in a depthward motion and uses a stack to remember to get the next vertex to start a search, when a dead end occurs in any iteration.



- As in the example given above, DFS algorithm traverses from A to B to C to D first then to E, then to F and lastly to G. It employs the following rules.
- **Rule 1** − Visit the adjacent unvisited vertex. Mark it as visited. Display it. Push it in a stack.
- **Rule 2** − If no adjacent vertex is found, pop up a vertex from the stack. (It will pop up all the vertices from the stack, which do not have adjacent vertices.)
- **Rule 3** − Repeat Rule 1 and Rule 2 until the stack is empty

Algorithm:

1. DFS (using adjacency matrix): (a) Mark the start vertex as visited and print it. (b) For every unvisited neighbour of the current vertex (scanning its row in the matrix), recursively perform DFS. (c) Backtrack when no unvisited neighbours remain.
2. BFS (using adjacency list): (a) Mark the start vertex as visited and enqueue it. (b) While the queue is not empty, dequeue a vertex, print it, and enqueue all of its unvisited neighbours, marking them visited. (c) Repeat until the queue is empty.

Functions used:

- def dfs_matrix(start):
- def bfs_list(start):

conclusion:

Thus, We have successfully implemented DFS and BFS graph traversal and and Represented a graph using an adjacency matrix to perform DFS and an adjacency list to perform BFS.

----------program----------

```
# Mapping location names to indices for matrix representation
locations = ['A', 'B', 'C', 'D']
location_index = {name: idx for idx, name in enumerate(locations)}

# ------------------------------
# DFS using Adjacency Matrix
# ------------------------------
# Graph represented as adjacency matrix
adj_matrix = [
    # A  B  C  D
    [ 0, 1, 1, 0],  # A
    [ 1, 0, 0, 1],  # B
    [ 1, 0, 0, 1],  # C
    [ 0, 1, 1, 0]   # D
]
def dfs_matrix(start):
    visited = [False] * len(locations)
    result = []
    def dfs(node):
        visited[node] = True
        result.append(locations[node])
        for neighbor in range(len(adj_matrix)):
            if adj_matrix[node][neighbor] == 1 and not visited[neighbor]:
```

```
            dfs(neighbor)
    dfs(location_index[start])
    return result
# ------------------------------
# BFS using Adjacency List
# ------------------------------
# Graph represented as adjacency list
adj_list = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'D'],
    'D': ['B', 'C']
}
def bfs_list(start):
    visited = set()
    queue = deque([start])
    result = []
    while queue:
        node = queue.popleft()
        if node not in visited:
            visited.add(node)
            result.append(node)
            for neighbor in adj_list[node]:
                if neighbor not in visited:
                    queue.append(neighbor)
    return result
```

```
# ------------------------------
# Run the algorithms
# ------------------------------
def menu():
    print("\n===== Menu =====")
    print("1. DFS")
    print("2. BFS")
    print("3. Exit")
while True:
  menu()
  choice = input("Enter your choice: ")
  if choice == '1':
    start_location = 'A'
    dfs_result = dfs_matrix(start_location)
    print("DFS Traversal (using adjacency matrix):", dfs_result)
  if choice == '2':
    start_location = 'A'
    bfs_result = bfs_list(start_location)
    print("BFS Traversal (using adjacency list):", bfs_result)
  if choice == '3':
    break
```

Practical 10

AIM:

Solve the problem of delivering pizzas from a shop to all nearby customer locations in the minimum possible total time, using an appropriate graph-based data structure, and compare two approaches — Prim's Algorithm and Kruskal's Algorithm.

PROBLEM STATEMENT:

A pizza shop receives multiple orders from several locations. Assume that one pizza boy is tasked with delivering pizzas to nearby locations, which are represented using a weighted graph, where the time required to travel from one location to another represents the weight of the edge connecting the corresponding nodes. Solve the problem of delivering a pizza to all customers in the minimum total time, using an appropriate data structure. Implement the solution using both Prim's Algorithm and Kruskal's Algorithm and compare the resulting delivery networks.

OBJECTIVE:

1. To understand the concept of a weighted graph and the Minimum Spanning Tree (MST).
2. To implement Prim's algorithm (vertex-based, greedy) to construct a Minimum Spanning Tree.
3. To implement Kruskal's algorithm (edge-based, greedy, using a disjoint-set/Union-Find structure) to construct a Minimum Spanning Tree.
4. To apply and compare both MST approaches on a real-world minimum-time delivery-network problem.

OUTCOME:

1. Students will be able to represent a weighted graph using an adjacency matrix / list.
2. Students will be able to implement Prim's algorithm and Kruskal's algorithm and determine the minimum spanning tree of a graph, along with its total cost.
3. Students will be able to compare the two MST algorithms in terms of approach, data structures used, and suitability for dense/sparse graphs.

SOFTWARE / HARDWARE USED:

- Programming Language: Python 3.x
- IDE: VS Code / PyCharm / Google Colab / Jupyter Notebook

THEORY:

1. Weighted Graph:

A weighted graph associates a numeric weight (here, the delivery time in minutes) with every edge, representing the cost of moving between two connected locations.

2. Minimum Spanning Tree (MST):

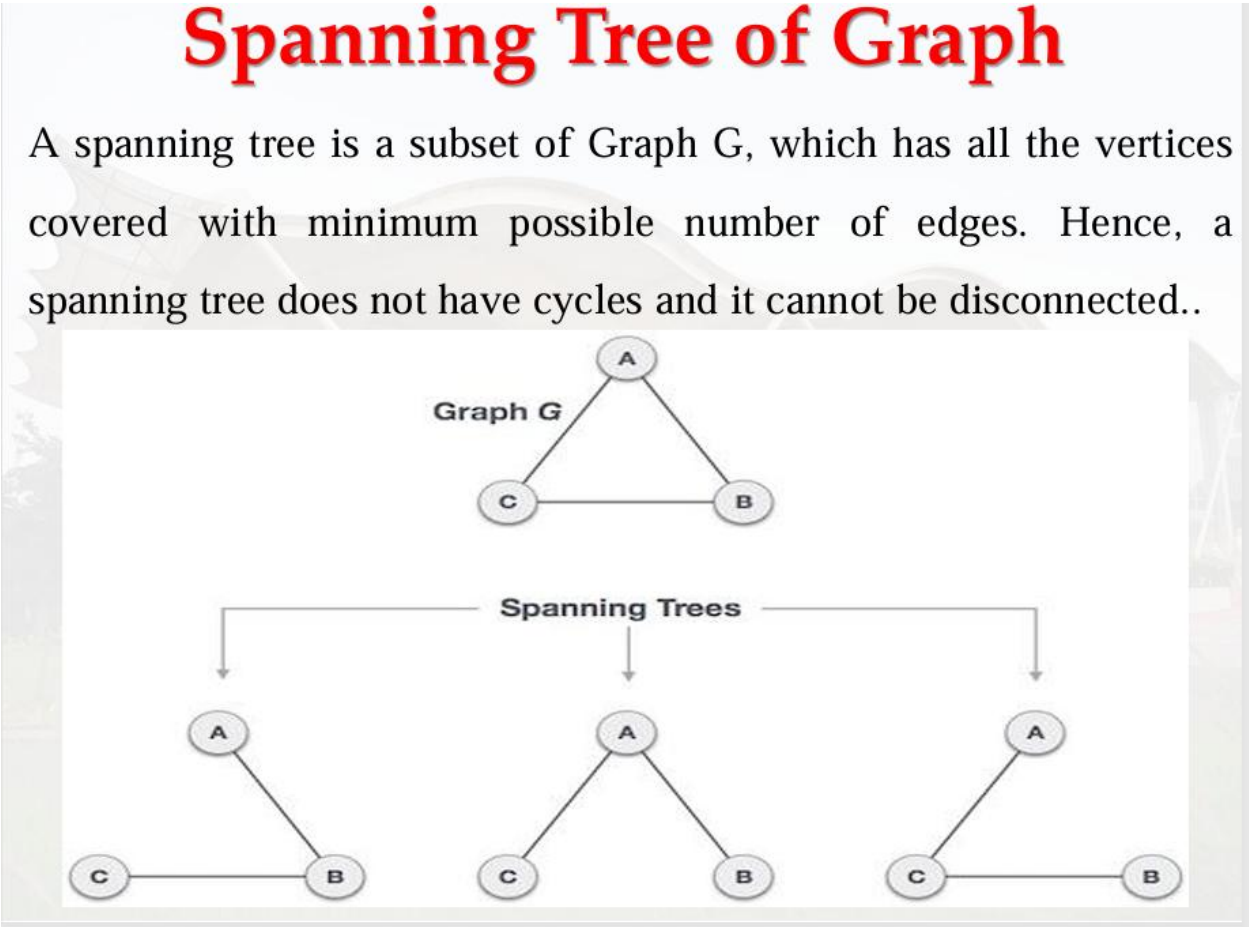
A spanning tree of a connected graph with V vertices is a subgraph that connects all the vertices using exactly V-1 edges and contains no cycle. A Minimum Spanning Tree is the spanning tree whose edges have the least possible total weight. Since the shop needs to reach every customer location while minimising the total travel/road-connection time, the delivery network that connects all locations at minimum total time corresponds exactly to the MST of the location graph.

3. Prim's Algorithm (Vertex-based approach):

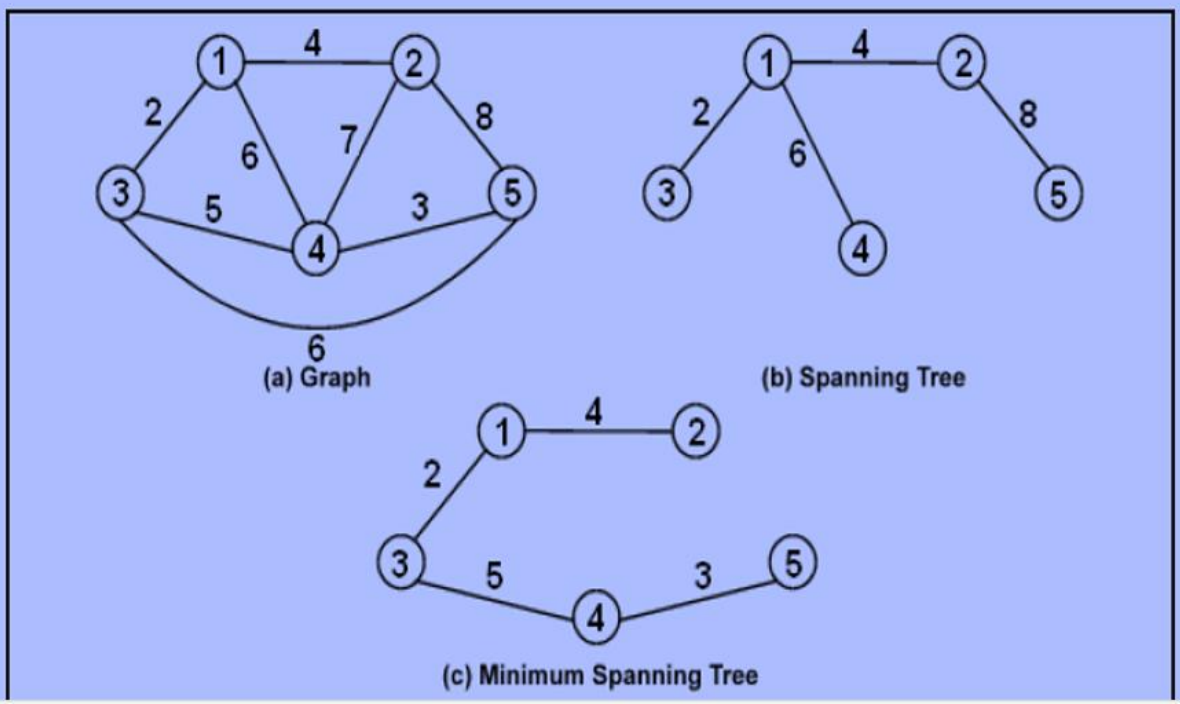
Prim's algorithm builds the MST greedily by growing a single tree one vertex at a time: starting from an arbitrary vertex, it repeatedly adds the minimum-weight edge that connects a vertex already in the tree to a vertex not yet in the tree, until all vertices are included. Using a min-priority queue (min-heap), Prim's algorithm runs in O(E log V) time and is well suited to dense graphs.

4. Kruskal's Algorithm (Edge-based approach):

Kruskal's algorithm builds the MST by considering all the edges of the graph in increasing order of weight, and greedily adding an edge to the MST only if it does not form a cycle with the edges already chosen. Cycle detection is done efficiently using a Disjoint-Set (Union-Find) data structure. Kruskal's algorithm runs in O(E log E) time (dominated by sorting the edges) and is well suited to sparse graphs.



Minimum Spanning Tree



GREEDY STRATEGIES

1. Kruskal's algorithms :

The steps for implementing Kruskal's algorithm are as follows:

1. Sort all the edges from low weight to high
2. Take the edge with the lowest weight and add it to the spanning tree. If adding the edge created a cycle, then reject this edge.
3. Keep adding edges until we reach all vertices.

GREEDY STRATEGIES

2. **Prims algorithm:** Prim's Algorithm also use Greedy approach to find the minimum spanning tree. In Prim's Algorithm we grow the spanning tree from a starting position. Unlike an **edge** in Kruskal's, we add **vertex** to the growing spanning tree in Prim's.

Algorithm Steps:

1. Initialize the minimum spanning tree with a vertex chosen at random.
2. Find all the edges that connect the tree to new vertices, find the minimum and add it to the tree.
3. Keep repeating **step** 2 until we get a minimum spanning tree.

Prim's Algorithm Complexity

The time complexity of Prim's algorithm is **O(E log V).**

Algorithm:

1. Represent the pizza shop and customer locations as vertices of a weighted graph, with edge weights equal to the travel time between two locations.
2. Prim's Algorithm: (a) Start with visited = {start_vertex} and a min-heap containing all edges from the start vertex. (b) Repeatedly pop the minimum-weight edge (u, v, weight); if v is already visited, discard it. (c) Otherwise add v to visited, add the edge to the MST, add its weight to the total, and push all edges from v to unvisited vertices onto the heap. (d) Repeat until every vertex has been visited.
3. Kruskal's Algorithm: (a) List all the edges of the graph and sort them in increasing order of weight. (b) Initialise a Disjoint-Set with every vertex in its own set. (c) For each edge (u, v, weight) in sorted order: if find(u) != find(v), accept the edge (add it to the MST, add its weight to the total, and union the two sets); otherwise discard the edge as it would form a cycle. (d) Repeat until V-1 edges have been added.
4. Compare the total minimum delivery time obtained from both algorithms — for a connected weighted graph with distinct edge weights, both algorithms must yield the same minimum total weight (though possibly a different valid MST if weights are not all distinct).

conclusion:

Thus, We have successfully implemented Minimum Spanning tree.

----------program----------

```
# Simple Kruskal's Algorithm for Pizza Delivery (Minimum Spanning Tree)

# Locations: Shop, A, B, C, D  -> represented as numbers 0,1,2,3,4
locations = ["Shop", "A", "B", "C", "D"]

# Each edge is written as [weight, location1, location2]
edges = [
    [4, 0, 1],   # Shop - A
    [8, 0, 2],   # Shop - B
    [2, 1, 2],   # A - B
    [5, 1, 3],   # A - C
    [3, 2, 3],   # B - C
    [7, 2, 4],   # B - D
    [1, 3, 4],   # C - D
]

n = len(locations)

# Step 1: Sort edges by weight (smallest first) using simple bubble sort
for i in range(len(edges)):
    for j in range(len(edges) - 1 - i):
        if edges[j][0] > edges[j + 1][0]:
            edges[j], edges[j + 1] = edges[j + 1], edges[j]

# Step 2: parent list to keep track of which group each location belongs to
parent = []
for i in range(n):
    parent.append(i)

# find the group leader of a location
def find_parent(loc):
    while parent[loc] != loc:
        loc = parent[loc]
    return loc

# join two groups together
def union(loc1, loc2):
```

```
    p1 = find_parent(loc1)
    p2 = find_parent(loc2)
    parent[p1] = p2

# Step 3: pick edges one by one, skip if it forms a cycle
total_time = 0
edges_used = 0

print("Minimum Time Delivery Network:")
for edge in edges:
    weight, u, v = edge
    if find_parent(u) != find_parent(v):
        union(u, v)
        total_time = total_time + weight
        edges_used = edges_used + 1
        print(locations[u], "->", locations[v], ":", weight, "min")
    if edges_used == n - 1:
        break

print("Total Minimum Delivery Time:", total_time, "min")
```

Output:

```
Minimum Time Delivery Network:
C -> D : 1 min
A -> B : 2 min
B -> C : 3 min
Shop -> A : 4 min
Total Minimum Delivery Time: 10 min
```